Adaptive Campaign Optimization Engine

DS 223 | Marketing Analytics | Group Project
Spring 2026 | American University of Armenia
Milestone 1 | *Problem Definition*

1. Problem Area

Campaign Personalization and Customer Conversion Optimization.

Marketing teams in e-commerce businesses commonly rely on static, rule-based campaign strategies: segment customers by RFM scores, assign a uniform promotion to each segment, and apply it broadly. While RFM provides a useful behavioral summary, it treats all customers within a segment identically and does not account for individual differences in how customers respond to different types of offers (Khajvand et al., 2011). These segments also reflect past behavior at a single point in time and do not update based on campaign outcomes. As a result, promotional budgets are not allocated optimally — some customers receive incentives they do not need, others receive the wrong type of offer, and potentially responsive customers may receive no intervention at all.

2. Preliminary Research

The limitations of static campaign allocation are well-documented in both academic and industry literature. The core issues are:

Uniform promotions reduce efficiency. Applying the same offer to an entire segment means subsidizing customers who would have converted regardless, while failing to provide the right incentive to those who need a different type of nudge. RFM segments group customers by behavioral similarity, but customers within the same segment may have diverse preferences and motivations (Omniconvert, 2025; Relay42, 2025).

A/B testing does not scale to many actions. Traditional A/B tests compare two variants with uniform random assignment. When the action space includes multiple offer types across diverse customer profiles, uniform randomization wastes budget on clearly suboptimal assignments and converges slowly.

Predictive models predict, but do not prescribe. Churn or propensity models identify *who is* at risk or likely to convert, but do not answer *which action* to take for each individual. The gap between prediction and decision remains unaddressed.

Contextual bandits close this gap. Li et al. (2010) introduced LinUCB for personalized news recommendation at Yahoo, demonstrating that contextual bandits outperform

context-free approaches by conditioning action selection on user features. The same framework applies directly to marketing: customer features are the context, promotional offers are the arms, and profit is the reward.

This project applies the explore-exploit framework covered in the course to a downstream marketing campaign optimization problem. A key advantage of bandit-based optimization is that it is computationally lightweight, requiring only linear algebra operations, making it feasible even for small teams without large infrastructure budgets.

References

Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). A Contextual-Bandit Approach to Personalized News Article Recommendation. *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pp. 661–670. ACM. https://arxiv.org/abs/1003.0146

Chu, W., Li, L., Reyzin, L., & Schapire, R. E. (2011). Contextual Bandits with Linear Payoff Functions. *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 208–214.

Auer, P. (2002). Using Confidence Bounds for Exploitation-Exploration Trade-offs. *Journal of Machine Learning Research*, 3, 397–422.

Khajvand, M., Zolfaghar, K., Ashoori, S., & Alizadeh, S. (2011). Estimating customer lifetime value based on RFM analysis of customer purchase behavior: Case study. *Procedia Computer Science*, 3, 57–63.

3. Specific Problem Statement

Given a stream of customers with observed behavioral and transactional features, select the best promotional action for each customer at each decision point, in order to maximize cumulative profit over time, while learning which actions work best for which customer profiles.

Unlike a standard prediction task, this system does not simply forecast who will convert — it decides *what to do* about each customer and learns from the outcomes. Each decision produces a measurable reward (profit = revenue minus action cost), and the system uses that feedback to refine future decisions.

4. Proposed Solution

4.1 Overview

We build a Campaign Optimization Engine that works as follows: when a customer arrives on the platform, the system reads their feature profile — recency, frequency, monetary value, basket diversity, average order size, and purchase regularity. Based on this profile, the system selects one promotional action from a configurable set. After the customer either

converts or does not, the system records the resulting profit (revenue minus the cost of the action) and updates its estimates of how effective each action is for that type of customer. Over time, the system learns to assign different actions to different customer profiles, concentrating budget where it has the highest expected return.

The core algorithm is LinUCB (Li et al., 2010), a contextual bandit that maintains a per-action estimate of expected reward given customer features. It balances exploration (trying actions it is uncertain about) with exploitation (choosing actions it believes are best), controlled by a single tunable parameter, alpha.

4.2 Data Strategy (Hybrid)

Real component (customer features): We use real data (for example, the UCI Online Retail II) which contains approximately 500,000 transaction records from a UK-based online retailer (2009–2011). From this raw transaction log, we engineer the following customer-level features:

Feature	How It's Computed
Recency	Days since last purchase
Frequency	Number of distinct orders
Monetary value	Average spend per order
Basket diversity	Number of unique product categories purchased
Average order size	Mean items per transaction
Purchase regularity	Standard deviation of inter-purchase intervals

These six features form the context vector that the model uses to make decisions.

Synthetic component (response simulation): We simulate how customers respond to promotional actions. Conversion probability is modeled as a logistic function over observed

features with action-specific coefficients, plus controlled noise. This approach is standard in bandit research — it is how the original LinUCB paper was evaluated (Li et al., 2010) — and is explicitly disclosed. No claim of real outcome data is made. The simulation allows us to control ground truth for rigorous evaluation.

4.3 Action Space

The system accepts any set of marketing actions as input. For our simulation, we define five actions that represent common promotional strategies with distinct cost-benefit profiles:

Action	Description	Cost	Expected Effect
A0: No action	Control group — no promotion sent	$0	Baseline conversion rate
A1: Discount (10%)	Targets price-sensitive customers	Margin reduction	Higher conversion, lower margin
A2: Free shipping	Removes friction for moderate-basket customers	Shipping cost absorbed	Moderate conversion lift
A3: Product rec.	Personalized suggestion based on category affinity	Minimal (compute)	Moderate lift at minimal cost
A4: Bundle offer	Cross-sell complementary items at slight discount	Small margin reduction	Higher basket value if converted

These were chosen because they differ in both cost structure and expected customer response, giving the bandit meaningful variation to learn from. The specific actions are configurable; the system's value lies in learning which action to assign to which customer, not in the actions themselves.

4.4 Analytical Technique

Primary model: LinUCB (Linear Upper Confidence Bound). For each action, the model maintains a ridge regression estimate of expected reward given the customer's feature vector. At each decision point, it selects the action with the highest estimated reward plus

an exploration bonus. The exploration bonus is large when the model is uncertain about an action's effect for a given customer profile, and shrinks as more data is collected. This naturally balances exploration and exploitation.

Baselines:

Random policy — uniform random action selection, representing no intelligence.
Greedy heuristic — rule-based assignment (e.g., always discount high-RFM users), representing traditional static segmentation.

We compare cumulative reward, per-step regret, and action diversity across all policies.

4.5 Implementation Plan (Microservice Architecture)

The system is implemented as a Dockerized microservice stack per course requirements:

Service	Technology	Responsibility	Container
Database	PostgreSQL	Store customers, actions, rewards, model state	db
ETL / Simulation	Python	Load real features, simulate conversion responses	etl
Model	Python (NumPy)	LinUCB engine, policy selection, parameter updates	model
Backend / API	FastAPI + SQLAlchemy	Serve decisions, log interactions, expose endpoints	api
Frontend	Streamlit	Dashboard: decisions, rewards, policy comparison	app

5. Expected Outcomes

LinUCB achieves higher cumulative profit than random and heuristic baselines over a simulated campaign horizon.

The system learns to differentiate actions across customer profiles (heterogeneous treatment effects), assigning different actions to different customer types.

Exploration decreases over time as the model gains confidence, visible in the action distribution converging.

A working MVP demonstrating the full decision loop: customer arrives, system selects action, outcome is observed, model updates, next decision improves.

6. Evaluation Metrics

Metric	Definition	Why It Matters
Cumulative Reward	Sum of (revenue − cost) over all decisions	Primary metric; directly measures profit
Per-Step Regret	Oracle policy reward minus chosen action reward	Measures learning speed; should decrease over time
Action Distribution	Proportion of each action selected per round	Shows exploration–exploitation balance
Conversion Rate by Action	Realized P(convert | action) over time	Validates the model learns response heterogeneity

7. Scope Boundaries

In Scope (MVP): 5 configurable actions, LinUCB + 2 baselines, single-step decision horizon, hybrid data (real features from UCI Online Retail II, synthetic responses), full Dockerized microservice stack with PostgreSQL, FastAPI, and Streamlit.

Out of Scope (Future Work): Temporal dynamics (customer fatigue, engagement decay), latent unobservable customer traits, multi-step sequential planning, real-world A/B deployment, deep learning models, and distributed data infrastructure.